

Introducing Kernel Samepage Merging



The motivation of KSM comes from the observation that on a large machine running several virtual machines (potentially running the same OS), there are a number of duplicate pages, each consuming a page of memory in the hypervisor.

The goal of KSM is to find and merge pages with similar content. Today, computer systems have a large number of resources at their disposal, but the most critical among them is the memory. A delay in acquiring memory, paging or swapping has a noticeable impact on performance. In fact, many production systems run with swap disabled to ensure maximum performance on the system.

While the trend is to have large memory built from higher density and cheaper devices with lower latency, the demand for memory bandwidth is also growing significantly due to the pervasiveness of multi-core machines. Another trend is to consolidate workloads into fewer machines to save space and power. KSM is a mechanism to increase the efficiency of consolidating applications by sharing pages.



Note: The term hypervisor is frequently used in the article; the reader can think of it as an operating system that can support virtualisation. The definitions we use in this article are not precise, but close enough to help understand the technology being referred to in this article.

The motivation for KSM

The motivation for KSM comes from the observation that on a large machine running

several virtual machines (potentially running the same OS), there are a number of duplicate pages, each consuming a page of memory in the hypervisor. If pages with exactly the same content don't change often, then sharing these pages across the various users will free up the memory for other use in the system. The ability of the hypervisor to provide more memory than is really available (via virtual memory or other means) is *memory overcommit*. The scope for sharing text regions (compiled code and instructions) is enormous. In a traditional operating system, text sharing happens automatically.

Figure 1 illustrates a set of applications (shown via their virtual address spaces) as seen by the memory management subsystem. In the case of the hypervisor, each of these applications can be a virtual instance of an operating system that's running.

Figure 2 shows how KSM merges pages with common content and replaces the pointers in each of the virtual address spaces to point to them. It is possible to make a rough estimate of the savings.

Let the number of pages with similar content in virtual address space i be n_i . The total scope for memory sharing is $\sum n_i$. The actual savings are $\sum n_i - n$, since we need to keep one copy of the shared pages. Note that this is a very simplistic calculation; in reality,